

COURSE CODE/TITLE:	CPE 010	SECTION:	CPE21S1
DATE PERFORMED:	July 16, 2026	DATE SUBMITTED:	July 21, 2026
NAME:	Mendoza, Ryo Shin P.	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. 2.1 **Array, Pointers, and Dynamic Allocation**

1. PROCEDURE OUTPUT

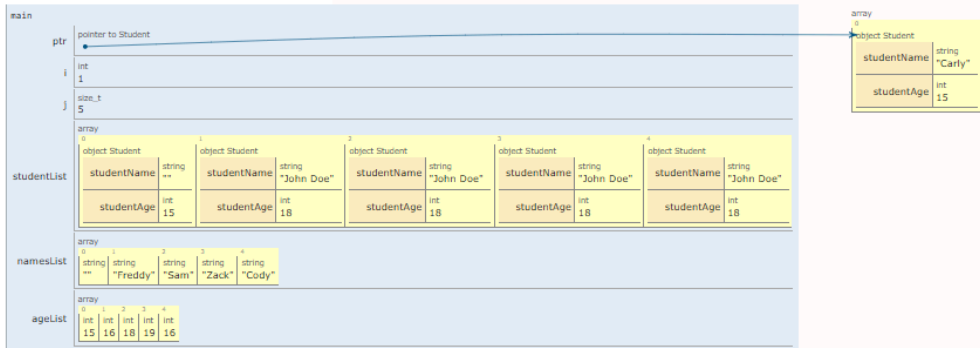
ILO A: Implement static and dynamic memory allocation & ILO B: Create dynamically allocated objects using pointers and arrays

OUTPUT	
Screenshot	C++ (C++20 + GNU extensions) <pre> 26 studentAge = copyStudent.studentAge, 27 } 28 29 // Copy Assignment Operator 30 Student& operator=(const Student &copyStudent) { 31 std::cout << "Copy Assignment Operator Called." << std::endl; 32 33 if (this != &copyStudent) { 34 studentName = copyStudent.studentName; 35 studentAge = copyStudent.studentAge; 36 } 37 38 return *this; 39 } 40 41 // Display Attributes 42 void printDetails() { 43 std::cout << studentName << " " << studentAge << std::endl; 44 } 45 }; 46 47 int main() { 48 49 Student student1("Roman", 28); 50 Student student2(student1); 51 Student student3; 52 student3 = student2; 53 54 return 0; 55 }</pre> Edit this code Print output (drag lower right corner to resize) <pre> Constructor Called. Copy Constructor Called. Constructor Called. Copy Assignment Operator Called. Destructor Called. Destructor Called. Destructor Called.</pre> Stack Heap <pre> main 0xFFFF00B80 object Student studentName 0xFFFF00B80 string "" studentAge 0xFFFF00B88 int 28 student1 0xFFFF00BB0 object Student studentName 0xFFFF00BB0 string "" studentAge 0xFFFF00BB8 int 28 student2 0xFFFF00BC0 object Student studentName 0xFFFF00BC0 string "" studentAge 0xFFFF00BC8 int 28</pre>
Observation	The program demonstrates the use of the constructor, copy constructor, and copy assignment operator. The constructor is called when objects are created, the copy constructor is called when student2 is initialized using student1, and the copy assignment operator is called when student3 is assigned the value of student2. Finally, the destructor is called automatically when the program ends.

Table 2-1. Initial Driver Program

Screenshot	<pre> 47 int main() { 48 49 const size_t j = 5; 50 Student studentList[j] = {}; 51 std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"}; 52 int ageList[j] = {15, 16, 18, 19, 16}; 53 54 return 0; 55 }</pre> Print output (drag lower right corner to resize) Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Destructor Called. Destructor Called. Destructor Called. Destructor Called. Destructor Called. Stack main j size_t 5 studentList array 0 1 2 3 4 object Student object Student object Student object Student object Student studentName studentName studentName studentName studentName studentAge studentAge studentAge studentAge studentAge namesList array 0 1 2 3 4 string string string string string ageList array 0 1 2 3 4 int int int int int
Observation	Based on my observation, five Student objects were created using static memory allocation. Since no student information was displayed, only the constructor and destructor messages appeared. The objects were automatically destroyed when the program ended.

Table 2-2. Modified Driver Program with Student List

Loop A	<pre> int main() { const size_t j = 5; Student studentList[j] = {}; std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"}; int ageList[j] = {15, 16, 18, 19, 16}; //Loop A for(int i = 0; i < j; i++){ Student *ptr = new Student(namesList[i], ageList[i]); studentList[i] = *ptr; } </pre>
Observation	Loop A dynamically allocates memory for each Student object using the new operator. Each object is initialized with a name and age, copied into the studentList array, and then the dynamically allocated memory is released using delete to avoid memory leaks.
Loop B	<pre> //Loop B for(int i=0;i<j;i++) { studentList[i].printDetails(); } </pre>
Observation	Loop B traverses the studentList array and calls the printDetails() function for each Student object. This displays the name and age of every student stored in the array.
Output	

	<pre> PS C:\Users\Shin\OneDrive\coding\DSA Activities\HOA 1.1\HOA 2.1> cd "c:\Users\Shin\OneDrive\coding\DSA Activities\HOA 1.1\HOA 2.1\" ; if (\$?) { g++ ILOA2-1.cpp -o ILOA2-1 } ; if (\$?) { .\ILOA2-1 } Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Copy Assignment Operator Called. Constructor Called. Copy Assignment Operator Called. Constructor Called. Copy Assignment Operator Called. Constructor Called. Copy Assignment Operator Called. Constructor Called. Copy Assignment Operator Called. Carly 15 Freddy 16 Sam 18 Zack 19 Cody 16 Destructor Called. Destructor Called. Destructor Called. Destructor Called. Destructor Called. </pre>
Observation	The program demonstrates both static and dynamic memory allocation. The Student objects were first stored in a statically allocated array, while temporary Student objects were dynamically created using the new operator. After copying the data into the array, the dynamically allocated memory was released using delete. Finally, the student information was displayed correctly using the printDetails() function.

Table 2-3. Final Driver Program

Modifications	<pre> for (int i = 0; i < j; i++) { Student *ptr = new Student(namesList[i], ageList[i]); studentList[i] = *ptr; delete ptr; } </pre> I inserted delete ptr; after copying each dynamically allocated object to prevent memory leaks.
Observation	The modifications improved memory management and ensured that dynamically allocated memory was properly released. The program executed correctly without memory leaks.

Table 2-4. Modifications/Corrections Necessary

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve programming problems using dynamic memory allocation, arrays and pointers

Jenna's Grocery List		
Apple	PHP 10	x7
Banana	PHP 10	x8
Broccoli	PHP 60	x12
Lettuce	PHP 50	x10

Jenna wants to buy the following fruits and vegetables for her daily consumption. However, she needs to distinguish between fruit and vegetable, as well as calculate the sum of prices that she has to pay in total.

Problem 1: Create a class for the fruit and the vegetable classes. Each class must have a constructor, destructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem description above.

Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List. You must then access each saved instance and display all details about the items.

Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List.

Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.

Problem 5: Discuss the

SOURCE CODE

```
#include <iostream>
#include <string>

//===== FRUIT CLASS =====
class Fruit {
private:
    std::string name;
    int price;
    int quantity;

public:
    // Constructor
    Fruit(std::string n = "", int p = 0, int q = 0) {
```

```
    name = n;
    price = p;
    quantity = q;
}

// Destructor
~Fruit() {
    std::cout << "Fruit " << name << " destroyed." << std::endl;
}

// Copy Constructor
Fruit(const Fruit& other) {
    name = other.name;
    price = other.price;
    quantity = other.quantity;
}

// Copy Assignment Operator
Fruit& operator=(const Fruit& other) {
    if (this != &other) {
        name = other.name;
        price = other.price;
        quantity = other.quantity;
    }
    return *this;
}

// Calculate total price
int calculateSum() {
    return price * quantity;
}

// Display information
void display() {
    std::cout << "Type: Fruit" << std::endl;
    std::cout << "Name: " << name << std::endl;
    std::cout << "Price: PHP " << price << std::endl;
}
```

```

        std::cout << "Quantity: " << quantity << std::endl;
        std::cout << "Total: PHP " << calculateSum() << std::endl;
        std::cout << "-----" << std::endl;
    }
};

//===== VEGETABLE CLASS =====
class Vegetable {
private:
    std::string name;
    int price;
    int quantity;

public:
    // Constructor
    Vegetable(std::string n = "", int p = 0, int q = 0) {
        name = n;
        price = p;
        quantity = q;
    }

    // Destructor
    ~Vegetable() {
        std::cout << "Vegetable " << name << " destroyed." <<
std::endl;
    }

    // Copy Constructor
    Vegetable(const Vegetable& other) {
        name = other.name;
        price = other.price;
        quantity = other.quantity;
    }

    // Copy Assignment Operator
    Vegetable& operator=(const Vegetable& other) {
        if (this != &other) {

```

```
        name = other.name;
        price = other.price;
        quantity = other.quantity;
    }
    return *this;
}

// Calculate total price
int calculateSum() {
    return price * quantity;
}

// Display information
void display() {
    std::cout << "Type: Vegetable" << std::endl;
    std::cout << "Name: " << name << std::endl;
    std::cout << "Price: PHP " << price << std::endl;
    std::cout << "Quantity: " << quantity << std::endl;
    std::cout << "Total: PHP " << calculateSum() << std::endl;
    std::cout << "-----" << std::endl;
}
};

//===== TOTAL SUM FUNCTION =====
int TotalSum(Fruit* fruits[], int fruitCount,
             Vegetable* vegetables[], int vegetableCount) {

    int total = 0;

    for (int i = 0; i < fruitCount; i++) {
        if (fruits[i] != nullptr) {
            total += fruits[i]->calculateSum();
        }
    }

    for (int i = 0; i < vegetableCount; i++) {
        if (vegetables[i] != nullptr) {
```



```
        total += vegetables[i]->calculateSum();
    }

}

return total;
}

//===== MAIN FUNCTION =====
int main() {

    // Grocery List
    Fruit* fruits[2];
    Vegetable* vegetables[2];

    // Create objects
    fruits[0] = new Fruit("Apple", 10, 7);
    fruits[1] = new Fruit("Banana", 10, 8);

    vegetables[0] = new Vegetable("Broccoli", 60, 12);
    vegetables[1] = new Vegetable("Lettuce", 50, 10);

    std::cout << "==== Jenna's Grocery List =====" << std::endl <<
std::endl;

    // Display Fruits
    for (int i = 0; i < 2; i++) {
        fruits[i]->display();
    }

    // Display Vegetables
    for (int i = 0; i < 2; i++) {
        vegetables[i]->display();
    }

    // Display Total Amount
    std::cout << "Total Amount to Pay: PHP "
        << TotalSum(fruits, 2, vegetables, 2)
```

```

        << std::endl << std::endl;

// Delete Lettuce
std::cout << "Deleting Lettuce..." << std::endl;

delete vegetables[1];
vegetables[1] = nullptr;

std::cout << std::endl;

// Display Updated Total
std::cout << "Updated Total: PHP "
        << TotalSum(fruits, 2, vegetables, 2)
        << std::endl << std::endl;

// Delete remaining objects
delete fruits[0];
delete fruits[1];
delete vegetables[0];

return 0;
}

```

Figure 1. Source Code

This program uses object-oriented programming by creating two classes, **Fruit** and **Vegetable**, to store the name, price, and quantity of grocery items. Each class includes a constructor to initialize the object's data, a destructor to display a message when an object is deleted, a copy constructor, and a copy assignment operator. The `calculateSum()` function computes the total cost of each item by multiplying its price and quantity, while the `display()` function shows the item's information. In the `main()` function, the program dynamically creates the grocery objects, displays their details, calculates the total amount using the `TotalSum()` function, deletes the `Lettuce` object to demonstrate memory deallocation, recalculates the total, and finally deletes the remaining objects before ending the program.

OUTPUT

```
===== Jenna's Grocery List =====
Type: Fruit
Name: Apple
Price: PHP 10
Quantity: 7
Total: PHP 70
-----
Type: Fruit
Name: Banana
Price: PHP 10
Quantity: 8
Total: PHP 80
-----
Type: Vegetable
Name: Broccoli
Price: PHP 60
Quantity: 12
Total: PHP 720
-----
Type: Vegetable
Name: Lettuce
Price: PHP 50
Quantity: 10
Total: PHP 500
-----
Total Amount to Pay: PHP 1370

Deleting Lettuce...
Vegetable Lettuce destroyed.

Updated Total: PHP 870

Fruit Apple destroyed.
Fruit Banana destroyed.
Vegetable Broccoli destroyed.
```

Figure 2. Output

The program first displays Jenna's grocery list, including the type, name, price, quantity, and total cost of each fruit and vegetable. It then prints the overall total amount by adding the total cost of all grocery items. After that, the program deletes the Lettuce object and displays the updated total amount without Lettuce's cost. Finally, the destructors are called as the remaining objects are deleted, displaying messages that confirm each object has been successfully destroyed before the program terminates.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.



COMPUTER
ENGINEERING

QUEZON CITY

Purpose of Use: Code Generator

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: I confirm that AI will not replace my critical thinking.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.